

HEXAWARE LMS PLATFORM

Trainer Dashboard

Backend Architectural Specification & API Design

A comprehensive implementation guide for database schema, model definitions, REST APIs, and UI-to-Backend integration workflows.

Document Version:	1.0.0 (Release-Ready)
Target Audience:	Backend & Database Engineering Teams
Tech Stack Focus:	FastAPI, SQLAlchemy ORM, PostgreSQL
Author:	Senior Technical Enablement Architect
Date:	July 2026

Table of Contents

1. Executive Summary & Purpose	3
2. System Architecture & Tech Stack	3
3. Role-Based Access Control (RBAC)	4
4. Database Schema Design (Proposed)	4
5. REST API Specifications	7
6. Frontend-Backend Integration Workflows	10
7. Implementation Checklist & Phases	11

Document Scope & Intent

This document serves as the direct technical blueprint for the backend team to build APIs and schema updates matching the newly-implemented frontend Trainer Dashboard. All APIs, field names, response formats, and workflows correspond 1-to-1 with the React components and data grids in the frontend application.

1. Executive Summary & Purpose

The Hexaware LMS Platform empowers senior trainers to monitor class progress, manage batches, evaluate coding assessments, schedule training sessions, and analyze course performance statistics. While the frontend prototype has been designed with interactive mockups, the backend server requires specific schema expansions and API endpoints to make these features active.

By implementing this technical specification, the backend team will enable full data persistence and business logic for the trainer workflow, allowing real-time grading, automated alerts for struggling students, and session tracking.

2. System Architecture & Tech Stack

The project adheres to a standard 3-tier architecture. This backend specification utilizes the existing technologies in the repository:

- **FastAPI:** Modern, high-performance web framework for building Python API services.
- **SQLAlchemy ORM:** Object Relational Mapper for PostgreSQL databases using async connection sessions.
- **Alembic:** Handles migrations for progressive schema evolution.
- **Pydantic v2:** Used for strict request body parsing, payload verification, and response serialization.
- **Uvicorn:** Asynchronous Server Gateway Interface (ASGI) running the FastAPI application server.

ARCHITECTURAL COMPLIANCE: Ensure all new routes are registered in the main router file (`app/main.py`) and obey the prefix guidelines (`/api/trainer`) to maintain consistency with existing client services.

3. Role-Based Access Control (RBAC)

Currently, the system schema (specifically the **users** table) does not feature user role segmentation. To secure trainer-only resources, we must define roles and implement authorization middleware.

Role Definitions (Enum)

A Python **Enum** should be declared to establish user privilege levels in `app/models/user.py` (or a core types file):

```
import enum

class UserRole(str, enum.Enum):
    TRAINEE = "trainee"
    TRAINER = "trainer"
    ADMIN = "admin"
```

Authorization Middleware Guidelines

A dependency function `require_role(allowed_roles: List[UserRole])` must be added to `app/core/security.py`. This function will read the JWT payload, query the current authenticated user, and verify if the user's role matches the allowed permissions before resolving the endpoint. Unauthorized attempts must throw an HTTP 403 Forbidden exception.

4. Database Schema Design (Proposed)

To support the Trainer Dashboard, several new tables must be added, and the users table updated. Below are the detailed specifications for SQLAlchemy models and database columns.

A. Update Model: User (Table: users)

Add the role column to the users table to map privileges.

Column Name	Type	Constraints	Description
role	VARCHAR(50)	NOT NULL, DEFAULT 'trainee'	Maps authorization access. Can be 'trainee', 'trainer', or 'admin'.

B. New Model: Batch (Table: batches)

Batches group trainees under a specific trainer and curriculum. Each batch corresponds to a single course.

Column Name	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Primary Key identifier

name	VARCHAR(255)	NOT NULL, UNIQUE	Cohort label e.g., 'Batch 2026 — Java Full-Stack'
course_id	INTEGER	FOREIGN KEY (courses.id), NOT NULL	Associated course curriculum
trainer_id	INTEGER	FOREIGN KEY (users.id), NOT NULL	Trainer user in charge of cohort
start_date	DATE	NULLABLE	Official commencement date
end_date	DATE	NULLABLE	Official conclusion date
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Indicates if the batch is current
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Record creation timestamp

C. New Association Model: BatchTrainee (Table: batch_trainees)

Associative link mapping users (trainees) to specific training cohorts.

Column Name	Type	Constraints	Description
batch_id	INTEGER	PRIMARY KEY, FK(batches.id)	Target Batch ID
trainee_id	INTEGER	PRIMARY KEY, FK(users.id)	Target Trainee User ID
joined_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Trainee insertion timestamp

D. New Model: Assignment (Table: assignments)

Specifies curriculum grading items (tasks, coding labs, quizzes) posted in a course.

Column Name	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Primary Key
title	VARCHAR(255)	NOT NULL	Name e.g., 'Assignment 3 — Exception Handling'
description	TEXT	NULLABLE	Detailed specifications and instructions
module_name	VARCHAR(255)	NOT NULL	Associated course section e.g. 'Core Java'
course_id	INTEGER	FOREIGN KEY (courses.id)	Curriculum assignment belongs to
max_score	INTEGER	NOT NULL, DEFAULT 100	Maximum achievable evaluation score
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Creation date

E. New Model: Submission (Table: submissions)

Stores raw trainee code/text responses, submission states, scores, and grading notes.

Column Name	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Primary Key
trainee_id	INTEGER	FOREIGN KEY (users.id)	Submitting trainee ID
assignment_id	INTEGER	FOREIGN KEY (assignments.id)	Associated assignment ID
submitted_code	TEXT	NOT NULL	Coding response or text answers
submitted_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Submission timestamp
status	VARCHAR(50)	NOT NULL, DEFAULT 'PENDING'	Submission status: 'PENDING' or 'GRADED'
score	INTEGER	NULLABLE	Assessed grade (0 to 100)

feedback	TEXT	NULLABLE	Trainer code-level review feedback
graded_by	INTEGER	FOREIGN KEY (users.id), NULLABLE	Evaluating Trainer User ID
graded_at	TIMESTAMP	NULLABLE	Evaluation completion time

F. New Model: LiveSession (Table: live_sessions)

Manages trainer-led training, lab interactive classes, and webinars.

Column Name	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Primary Key
title	VARCHAR(255)	NOT NULL	Lecture/Workshop topic label
description	TEXT	NULLABLE	Syllabus details or prep instructions
session_type	VARCHAR(50)	NOT NULL	Can be 'LIVE_SESSION', 'WORKSHOP', or 'REVISION'
batch_id	INTEGER	FOREIGN KEY (batches.id)	Target trainee cohort
trainer_id	INTEGER	FOREIGN KEY (users.id)	Host trainer user ID
start_time	TIMESTAMP	NOT NULL	Target execution start time
end_time	TIMESTAMP	NOT NULL	Target execution end time
meeting_link	VARCHAR(1000)	NULLABLE	Virtual classroom URL e.g. Teams, WebEx
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Session insertion timestamp

G. New Model: AttendanceRecord (Table: attendance_records)

Tracks individual attendance indicators for scheduled live classes.

Column Name	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Primary Key
trainee_id	INTEGER	FOREIGN KEY (users.id)	Target student user ID
session_id	INTEGER	FOREIGN KEY (live_sessions.id)	Scheduled session reference
status	VARCHAR(50)	NOT NULL	Attendance: 'PRESENT', 'ABSENT', or 'LATE'
marked_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Trainee mark registration timestamp

DATABASE MIGRATION WARNING: Alembic migrations must be generated sequentially. Make sure to back up test data prior to running the upgrade script in the staging environment.

5. REST API Specifications

The backend must implement the following REST endpoints to serve data to the dashboard and handle updates. All routes are protected by the RBAC middleware and require a valid Bearer token.

A. Module 1: Dashboard Overview Metrics

Aggregates key statistics, live session countdown, and scheduled calendars for the current trainer.

Attribute	Details
Method / Path	GET /api/trainer/overview
Description	Gathers aggregate summary statistics for the dashboard homepage including trainee count, active cohorts, assignments pending grading, the closest upcoming session, and lists the 4 nearest upcoming classes.
Request Payload	None (JWT authenticated)
Response Schema	{ "totalTrainees": 48, "activeBatches": 3, "pendingGrades": 7, "nextSessionISO": "2026-07-13T10:00:00Z", "upcomingSessions": [{ "id": "s1", "title": "Core Java – OOP Fundamentals", "type": "Live Session", "batch": "Batch 2026 - Java Full-Stack", "date": "Mon, 07 Jul 2026", "time": "10:00 AM – 11:30 AM", "colorClass": "session-blue", "icon": "video" }] }

B. Module 2: Batch & Trainee Management

Allows a trainer to view details of their assigned student cohorts and trainee performance indicators.

Attribute	Details
Method / Path	GET /api/trainer/batches
Description	Retrieves a lists of all batches / courses assigned to the logged-in trainer.
Request Payload	None (JWT authenticated)
Response Schema	[{ "id": "batch-java", "label": "Batch 2026 – Java Full-Stack", "course": "Core Java Full-Stack", "traineeCount": 6 }]

Fetch specific batch student progress reports.

Attribute	Details
Method / Path	GET /api/trainer/batches/{batch_id}/trainees
Description	Fetches a list of all trainees registered inside a specific batch, complete with academic progress percentage, attendance percentages, and curriculum status tags.
Request Payload	Query params: none. Path parameter: batch_id
Response Schema	[{ "id": "t1", "name": "Ananya Sharma", "email": "ananya.sharma@hexaware.com", "employeeId": "HEX-E-1021", "initials": "AS", "color": "#3563e9", "progressLabel": "Core Java", "progressPct": 78, "attendancePct": 94, "status": "On Track" }]

C. Module 3: Grading Queue & Assessment Hub

Handles code evaluation queue and grading responses submission.

Attribute	Details
Method / Path	GET /api/trainer/grading-queue
Description	Fetches assignments that have been submitted by trainees in the trainer's batches and are pending assessment. Includes the raw submitted code or text answers.
Request Payload	None (JWT authenticated)
Response Schema	[{ "id": "g1", "traineeName": "Rohan Mehta", "employeeId": "HEX-E-1034", "initials": "RM", "color": "#10B981", "module": "Core Java", "taskTitle": "Assignment 3 – Exception Handling", "submittedDate": "04 Jul 2026", "submittedCode": "public class BankAccount { ... }" }]

Submit grading evaluation payload.

Attribute	Details
Method / Path	POST /api/trainer/submissions/{submission_id}/grade
Description	Submits assessment metrics (numeric score out of 100, written qualitative feedback) to evaluate and close a pending task queue item.
Request Payload	<pre>{ "score": 85, "feedback": "Outstanding implementation of custom bank exception cases." }</pre>
Response Schema	<pre>{ "status": "success", "message": "Grade logged successfully.", "gradedAt": "2026-07-06T15:55:00Z" }</pre>

D. Module 4: Session Scheduler

Enables scheduling and deletion of live lectures and virtual classes.

Attribute	Details
Method / Path	POST /api/trainer/sessions
Description	Creates a new lecture, workshop, or revision session for a specific trainee batch.
Request Payload	<pre>{ "title": "Docker Containers Basics", "sessionType": "LIVE_SESSION", "batchId": "batch-cloud", "startTime": "2026-07-10T15:00:00Z", "endTime": "2026-07-10T17:00:00Z", "meetingLink": "https://teams.microsoft.com/..." }</pre>
Response Schema	<pre>{ "id": "s5", "title": "Docker Containers Basics", "sessionType": "LIVE_SESSION", "batchId": "batch-cloud", "startTime": "2026-07-10T15:00:00Z", "endTime": "2026-07-10T17:00:00Z", "meetingLink": "https://teams.microsoft.com/..." }</pre>

E. Module 5: Performance Analytics

Provides average class progress grades and triggers alerts for lagging concept areas.

Attribute	Details
Method / Path	GET /api/trainer/analytics/modules?batch_id={batch_id}
Description	Returns the average score of all trainees in the specified batch, broken down by course module, to draw bar charts.
Request Payload	Query parameter: batch_id (string)
Response Schema	[{ "id": "pm1", "name": "Java Basics & Syntax", "avgScore": 82 }, { "id": "pm2", "name": "OOP Fundamentals", "avgScore": 74 }, { "id": "pm3", "name": "Exception Handling", "avgScore": 68 }, { "id": "pm4", "name": "Collections Framework", "avgScore": 61 }]

Fetch diagnosis analytics alerts.

Attribute	Details
Method / Path	GET /api/trainer/analytics/alerts?batch_id={batch_id}
Description	Identifies weak course areas by scanning for modules with average scores under the 65% benchmark.
Request Payload	Query parameter: batch_id (string)
Response Schema	[{ "id": "pm4", "name": "Collections Framework", "avgScore": 61, "alertType": "WEAKNESS_IDENTIFIED", "recommendation": "Review sessions recommended" }]

6. Frontend-Backend Integration Workflows

To ensure a smooth transition from static screens to backend-driven workflows, developers should implement integration steps for key features as outlined below.

A. Real-Time Countdown for Next Session

The dashboard features a live countdown ticker showing the hours, minutes, and seconds until the next session starts. The logic is handled as follows:

- The backend exposes ``nextSessionISO`` (e.g. ``2026-07-13T10:00:00.000Z``) inside the ``GET /api/trainer/overview`` response payload.
- The React frontend (in ``TrainerOverview.jsx``) sets up a 1-second interval counter.
- On each tick, the current client datetime is subtracted from the backend-provided ISO string.
- If the difference is greater than zero, the components displays formatted ``HH:MM:SS`` hours, minutes, and seconds remaining.
- If the counter falls below zero, the UI updates its state to display 'Session in progress / ended' and the ticking loop stops.

B. Grading Submission Pipeline

When evaluating trainee code assignments, the following synchronization process occurs between client and server:

- Trainer navigates to the 'Grading Queue' tab. The UI calls ``GET /api/trainer/grading-queue`` and populates the left panel list.
- Trainer clicks a student's card. The React state caches the selection and opens the assessment review panel on the right, rendering the code syntax.
- Trainer enters a score (0-100) and writes detailed code feedback in the form controls, then clicks 'Submit Assessment'.
- The frontend sends a ``POST`` request to ``/api/trainer/submissions/{submission_id}/grade`` carrying the score and feedback payload.
- On a successful HTTP 200 OK server response, the React client removes the graded item from its state list, clears selection fields, and decrement the aggregate global counter by 1. No full page reload is triggered.

C. Diagnostic Analytics Refresh

The Performance Analytics visualizer updates dynamically when switching courses:

- When the dashboard loads, the frontend calls the modules API with the default batch ID ('java').
- The server executes an aggregate SQL query: it finds all graded submissions in that batch, filters by assignment module, and averages the scores. It returns the list to the client.
- Simultaneously, the frontend calls the alerts API to fetch struggling concepts (averages < 65%).
- When the trainer changes the dropdown to 'Cloud Architecture', a React ``onChange`` handler updates the ``selectedBatch`` state. This triggers dual API calls with the new batch ID, refreshing the graphical bar chart and

diagnostic sidebar instantly.

7. Implementation Checklist & Phases

To ensure structured development, backend implementation is divided into three consecutive phases. Progress can be validated against the criteria listed below.

Phase 1: Database Migration & Schema Setup

- Define `UserRole`` enum and add `role`` column to `User`` model (`app/models/user.py``).
- Create new SQLAlchemy models for `Batch``, `BatchTrainee``, `Assignment``, `Submission``, `LiveSession``, and `AttendanceRecord``.
- Generate Alembic migration script using: `alembic revision --autogenerate -m "add_trainer_models"`.
- Run migrations against Postgres testing server and inspect created tables to confirm foreign keys and cascade rules.

Phase 2: Authorization & Router Integration

- Create JWT role extraction logic in security helpers.
- Set up router prefixes and route definitions inside `app/routers/trainer.py``.
- Apply authorization dependency: `Depends(require_role([UserRole.TRAINER, UserRole.ADMIN]))` globally on the trainer router.
- Register the new trainer router in `app/main.py`` under the `/api/trainer`` prefix path.

Phase 3: Core API Endpoints & Business Logic

- Write repository queries for trainer overview stats (aggregated user, batch, and pending sub counts).
- Implement the grading submission endpoint, adding transaction handling to update submission rows and log status.
- Build the session creation route and connect validation checks to ensure sessions do not double-book a trainer's calendar.
- Develop analytics endpoints using SQLAlchemy grouping/averaging clauses to pull average module performance.

VERIFICATION TIP: For manual integration verification, developers can run the server locally, authenticate with a trainer user profile, and query endpoints using the interactive Swagger UI at <http://localhost:8000/docs>.